

一篇關於軟體、機器與遺忘的長文

程式碼變便宜之後

Vibe coding 與規格驅動開發吵了兩年。

一個在兩個時代都寫過程式的人,把這場架看完了,
想告訴你最後留在桌上的是哪些東西。

寫於二〇二六年八月

本文整理自一份軟體團隊分工研究報告的延伸討論

一 程式碼曾經是貴的

我入行的時候,程式碼是貴的。貴到什麼程度?一個資深工程師一天寫兩百行,大家覺得他很能幹。因為貴,我們發明了一整套儀式來保護它:需求規格書、概要設計、詳細設計、評審會議、簽核欄。文件比程式先寫,主管比編譯器先看。一個欄位要改名,得先開會。

後來程式碼慢慢變便宜,那套儀式開始被嘲笑。敏捷宣言發表那年,我正好在一個瀑布式專案裡加班,看著三百頁沒人讀完的規格書,和一個永遠跟規格書對不上的系統,心想這些人說得對,文件不該是這樣用的。

又過了二十幾年,程式碼便宜到接近免費。你對著一個聊天視窗描述想要什麼,幾分鐘後它就在你面前跑起來。Karpathy 在二〇二五年初給這種工作方式取了個名字,叫 `vibe coding`:順著感覺走,別細讀程式碼,全交給模型,錯了就再叫它改一次。

然後我看到兩群人吵起來了。一群人說,規劃已死,探索萬歲,寫規格的時間夠我迭代三輪。另一群人舉著「規格驅動開發」的旗子說,程式碼既然變便宜了,規格才是真正的資產,程式碼不過是規格的編譯產物。工具也跟著站隊:一邊是各種越來越順手的對話式編輯器,另一邊是 Kiro、Spec Kit 這類把規格放在正中央的框架。

我坐在旁邊聽了很久,越聽越覺得熟悉。這場架我年輕時看過一次,只是那時吵的雙方都是人,現在中間多了一台機器。而那台機器的脾氣,恰好決定了這次誰對誰錯。

二 老架新吵

先把舊帳翻出來,因為不翻舊帳,就看不懂新帳。

敏捷宣言的原文是「可用的軟體,重於詳盡的文件」。注意那個「重於」,它是一個比較級,不是一個否定句。宣言的作者們後來花了二十年到處澄清:我們沒有說不要文件。可是沒用。人們記住的版本是「敏捷等於不寫文件」,因為這個版本比較省力。於是接下來的二十年,整個行業在兩個極端之間擺盪:一邊是文件多到沒人讀,一邊是文件少到沒人知道系統為什麼長這樣。

Vibe coding 和規格驅動開發的爭論,是同一場擺盪,換了佈景重演。vibe 派繼承了「不寫文件」的誤讀,把 prompt 當成用完即丟的東西;規格派繼承了瀑布時代的鄉愁,夢想一份完美的規格能讓程式碼自動長出來。雙方都拿對方的失敗案例當武器,而且雙方的彈藥都很充足,因為對方確實常常失敗。

有意思的地方在這裡:這次的爭論有裁判。上一輪瀑布對敏捷,吵的是「人類團隊怎麼協作」,公說公有理,最後靠潮流分勝負。這一輪不一樣。這一輪的核心當事人是模型,而模型的能力和缺陷是可以觀察、可以重複驗證的。你不需要站隊,你只需要看清楚那台機器到底會在哪裡跌倒。

三 兩邊都說對了一半

Vibe 派說對的事,規格派不太願意承認:前期設計多半是猜的。我畫過的架構圖裡,能活過第一次真實流量的不到一半。程式碼昂貴的年代,猜錯的代價是幾個月,所以我們用大量的評審去降低猜錯的機率;程式碼便宜之後,做出來再改,常常真的比想清楚再做快。探索的價值被重新定價了,這是事實,抵抗它沒有意義。

Vibe 派說錯的事,他們自己在第二週就會發現:玩具和產品之間,不是規模的差異,是性質的差異。一個週末做出來的原型,和一個要維運三年、有真實使用者資料在裡面的系統,中間隔著的不是「再多 vibe 幾天」,而是一整套他們宣稱不需要的東西。很多人是在對著一坨自己看不懂的程式碼、叫模型「修好它」而它越修越壞的那個晚上,才明白這件事的。

規格派說對的事,任何用模型寫過兩週以上程式的人都體會過:模型會忘。它沒有跨對話的記憶,你上週和它辛苦對齊的架構決策,這週的它一無所知。沒有外部化的規格和決策紀錄,每一次生成都是重新擲骰子。「第二週崩潰」不是修辭,是可以預測的物理現象。

規格派說錯的事,或者說還沒想清楚的事:規格寫到和程式碼一樣細的時候,你只是換了一種語言在寫程式,而且是一種沒有編譯器、沒有型別檢查、歧義滿地的語言。瀑布時代我們就試過用文件窮盡一切,失敗的原因不是文件不夠多,是世界變得比文件快。這個原因今天依然成立。

所以真正的問題從來不是「要不要文件」。是哪些文件在治真實的病,哪些文件只是在治焦慮。

四 裁判不是人,是機器的病

要回答這個問題,得先把那台機器的病歷攤開。我這兩年看模型寫程式,看出它幾種很穩定的毛病。

它會忘。前面說過了,這是結構性的,不是它不用心。

它會漂。同一份需求,今天生成和明天生成,架構風格可以完全不同。沒有約束的時候,它不會維持一致性,它只會維持「局部看起來合理」。

它會編造。不存在的函式庫、不存在的 API 參數,它寫起來語氣和真的一模一樣。它不是在騙你,它只是分不清「記得」和「覺得應該有」。

它寫的測試,有時只是把實作抄一遍再宣告通過。看起來覆蓋率很漂亮,實際上什麼都沒驗證。

它會順手多做。你要一個登入頁,它送你一套會員系統,像個過度熱心的新人。範圍在你沒注意的時候悄悄變了。

最麻煩的是最後一條:它錯得很整齊。人類犯錯是零星的、東一個西一個;模型犯錯是成批的、風格一致的,而且披著一層「看起來很對」的外衣。它生產「貌似正確」的能力,遠遠強過任何一個人類工程師,這讓傳統的逐行 review 幾乎失效,因為逐行看,每一行都很有道理。

把這份病歷放在桌上,再回頭看那場爭論,判準就出來了:凡是治這些病的東西,砍不掉;凡是只治「人類協作病」的東西,在只有你和一台機器的房間裡,可以先放下。十三種職稱、每週十種例會、RACI 矩陣,那些是人多了以後的產物,治的是人與人之間的誤解和推諉。它們不是不重要,但它們不是本質。本質是剩下來的这一小把東西。

我把那一小把東西數了一遍。三頂帽子,四份文件,五個節點。

五 三頂帽子

先說清楚:是三頂帽子,不是三個人。一個獨立開發者自己全戴,一個大公司可以拆成幾十人,但帽子就是這三頂,而且有一個共同點,它們都不能戴在機器頭上。

第一頂,對意圖負責的人。決定解什麼問題、更重要的是決定不解什麼。模型的病歷裡有一條是「順手多做」,範圍會悄悄膨脹,所以必須有一個人對邊界有最後裁決權。vibe coder 其實一直在扮演這個角色,他每敲一句 prompt 都在行使這個權力,只是他沒意識到自己戴著帽子,也就沒意識到這頂帽子需要認真對待。

第二頂,對決策負責的人。技術方向、取捨、邊界,選了什麼,為什麼,不選什麼。機器會忘也會漂,所以決策必須被外部化成它每次都能重新讀到的東西,而外部化這個動作,得有人負責做、負責維護、負責在決策改變時更新。

第三頂,對驗證負責的人。判定「做完了沒有」。這頂帽子有一條鐵律:寫的人不能同時是驗收的人。這條規矩比人工智慧老得多,銀行對帳是這樣,論文審稿是這樣,四眼原則是這樣。模型可以幫你跑測試、幫你分析風險,但宣告「可以上線」的那個簽名,必須是人的,因為只有人可以承擔後果。責任這個東西,目前為止還無法生成。

你可能注意到了,傳統團隊裡那一長串角色,產品經理、分析師、架構師、測試、維運,全部可以塞進這三頂帽子裡。它們是三頂帽子在組織變大之後的裂解,不是新增的本質。

六 四份文件

然後是文件。過濾的標準只有一條:這份文件是寫給誰讀的?

過去的答案是:寫給未來的人。給下一個接手的工程師、給稽核、給法務、給三年後想知道當初為什麼這樣做的自己。這類文件的悲劇在於,未來的人常常不來,於是文件死在資料夾裡。

現在多了一個新讀者,而且這個讀者每天都來:下一次生成時的模型。文件不再是歸檔,是餵進上下文的記憶。用這個標準過濾,留下來的是四份。

- **意圖文件**。問題是什麼、目標是什麼、什麼在範圍外、怎樣算成功。vibe coder 本來就在寫這份文件,他寫在 `prompt` 裡,然後隨手丟掉。把丟掉的動作改成存下來,prompt 就變成了規格。兩派在這裡的距離,其實只有一個「另存新檔」。
- **決策日誌**。選了什麼、為什麼、不選什麼。治的是失憶和漂移。這份文件在傳統世界叫 `ADR`,在新世界叫 `CLAUDE.md`、`AGENTS.md`,或者規格派口中的 `constitution`。名字不同,本質完全一樣:讓每一次生成都被同一組決策約束。

- **介面契約**。API 的形狀、資料的形狀、模組之間的邊界。治的是幻覺和並行漂移。當幾個模型代理同時在一個系統的不同角落工作,契約是唯一能讓它們不互相踩死的東西。這是舊時代的凍結機制裡,唯一原封不動活下來的部分,而且因為契約可以生成 mock 和測試,它對機器的槓桿比對人更大。
- **可執行的驗收**。測試,加上白紙黑字的完成標準。這是兩派共識最高的一份,原因很務實:vibe coder 需要它,才能不讀程式碼而依然睡得著;規格派需要它,才能宣稱規格被滿足了。測試是唯一機器可以自己檢查的規格,是自然語言和程式碼之間僅存的一座硬橋。

其他的呢?利害關係人地圖、高保真設計規格、會議紀錄、責任分派矩陣,那些是「人多了才需要」的產物。組織要付的規模稅,不是軟體的本質成本。稅可以隨團隊大小增減,本質不行。

七 五個節點

最後是流程。這裡要先擋掉一個誤解:留下來的不是五個「階段」,是五個「節點」。階段是瀑布的語言,意思是上一段沒完成下一段不准動;節點是另一種東西,節點之間你要怎麼並行、怎麼探索、怎麼 vibe,悉聽尊便,但節點本身跳過去,代價會在後面等你,連本帶利。

生成之前,把意圖說清楚。哪怕只花十分鐘:解什麼,不解什麼,怎樣算成功。省下這十分鐘的下場,是模型用接下來十個小時去猜,而且猜得很有信心。

鋪開之前,把方向定下來。寫一則決策紀錄的時間就夠。太晚定方向的代價在機器時代被放大了:以前一個人走錯路,錯誤蔓延的速度是一天幾百行;現在模型走錯路,一夜之間錯誤就鋪滿整個程式庫,而且鋪得非常均勻、非常一致,清起來比人寫的爛攤子更費勁。

並行之前,把契約凍住。前後端分頭走之前、幾個代理分頭走之前,介面先穩。舊報告裡有一句話,說凍結是為了讓並行不失控。把句子裡的主詞從「團隊」換成「代理」,一個字都不用改。

宣稱完成之前,拿證據出來。用證據決策,不用氣氛決策。這條在人類時代是好習慣,在機器時代是生存條件,因為機器製造氣氛的能力太強了,它每一次交付看起來都很完成。

上線的時候,留好退路。可回滾、可觀測。模型寫的程式,壞法和人不一樣,它會犯人類不會犯的錯,錯在你根本沒想到要檢查的地方。退路是你 vibe 得起的保險費。

澄清、決策、凍結、驗證、退路。五個節點,一個都省不掉,但每一個都可以很輕。輕重隨專案調,有無不容商量。

八 最安靜的和解

寫到這裡,可以把這場爭論的結局說破了。它不會有贏家,因為兩邊正在變成同一種人,只是他們自己還沒發現。

我看過不少宣稱「純 vibe」的開發者的工作目錄。有意思的是,越是認真的那種,目錄裡越會躺著一個檔案,叫 CLAUDE.md 也好,叫 AGENTS.md 也好,裡面寫著專案的約定、架構的決策、模型不准碰的東西。他們管這個叫「調教」,叫「上下文工程」。我年輕的時候,我們管這個東西叫規格書和架構決策紀錄。

反過來,規格派的工具也在退讓。最早版本恨不得規格寫滿才准生成,現在一個個都加上了「先探索、後補規格」的模式,因為使用者用腳投票告訴他們:探索的自由才是這個時代最大的紅利,誰擋著紅利,誰被卸載。

一邊默默把 prompt 存成了文件,一邊默默把文件鬆綁成了對話。這是我職業生涯裡看過最安靜的一次和解。沒有人簽宣言,沒有人開大會,兩支軍隊在戰場上走著走著,穿成了同一身衣服。

而這次和解裡藏著一個真正的反轉,值得單獨說:**角色被壓縮了,流程被縮短了,文件的地位卻上升了**。過去三十年,每一波方法論革命都在把文件往下踩,文件從主角變配角,從配角變累贅。這一次反過來。因為文件換了讀者,它從寫給「可能永遠不會來的未來人類」,變成寫給「每天早上都準時報到的機器」。一份沒人讀的文件是死的;一份每次生成都被讀取的文件,是活的基礎設施。文件第一次有了心跳。

九 對抗遺忘

最後說點大的,反正文章要結束了,說大話的成本很低。

回頭看,軟體工程這門手藝,從頭到尾都在對抗同一個敵人:遺忘。需求會被忘記,所以有規格;決策會被忘記,所以有紀錄;教訓會被忘記,所以有測試,測試是把教訓釘在牆上的方式;人會離職,所以有交接文件。整棟方法論的大廈,地基就是這四個字:人會忘記。

現在來了一個不會累的同事。它聰明、勤快、寫字比誰都快,而且便宜得驚人。只有一個問題:它每天早上都是新來的。昨天發生的一切,它一概不在場。

你留給它什麼,它就是什麼。你什麼都不留,它就每天重新猜一次你要什麼,今天猜得和昨天不一樣,還猜得理直氣壯。你把意圖、決策、契約、驗收留下來,它就接得住,而且接得比任何人類新人都快。

所以那些吵架的人,吵的其實不是方法論。他們在吵的是:對一個沒有記憶的天才,你打算留下什麼。

三頂帽子,四份文件,五個節點。我入行三十年,看著程式碼從一天兩百行的奢侈品,變成一晚上兩萬行的日用品,價格跌了不知道幾萬倍。而這一小把東西的價格,一毛錢都沒跌。

大概,這就是本質這個詞的意思。

附記:本文的分析框架來自一份關於軟體產品團隊分工的研究報告。該報告描述了完整企業規模下的十三種角色、十二項交付物與十一個階段;本文所做的,是用 `vibe coding` 與規格驅動開發兩派互相指出的痛點、以及模型寫程式的實際故障模式作為濾網,從中過濾出無法再壓縮的最小集合。兩份文本一厚一薄,說的是同一件事。